

P2P Implementation of Distributed Algorithms

Raswanth J

June 2024

1 Introduction

The objective of this project is to design and implement a distributed consensus algorithm using a network of Raspberry Pi devices. The project specifically focuses on achieving consensus among multiple nodes through a coordinated approach, where a central coordinator initiates the process. The primary aim is to enable each node to reach an average consensus value through iterative communication with its neighbors.

The consensus algorithm implemented in this project is a fundamental problem in distributed computing, where the goal is to have all nodes in a distributed system agree on a certain value or state. This is particularly important in systems where reliability, fault tolerance, and consistency are critical. By using Raspberry Pi devices, this project demonstrates a practical approach to solving the consensus problem in a distributed network.

Distributed Algorithms are a class of algorithms specifically designed to operate in distributed computing environments. In such environments, multiple computing nodes (which can be physically separated) communicate and coordinate with each other to achieve a common goal. These algorithms are integral to distributed systems, where the nodes work collectively to solve problems that are beyond the capabilities of a single machine.

Key Characteristics of Distributed Algorithms:

1. **Concurrency:** Multiple nodes perform computations simultaneously.
2. **Communication:** Nodes exchange messages to coordinate actions and share information.
3. **Fault Tolerance:** The system can continue to function even if some nodes fail.
4. **Scalability:** The system can handle an increasing number of nodes without significant performance degradation.
5. **Decentralization:** No single node has complete control; instead, control and data are distributed across nodes.

2 Goals

1. Single File Implementation: Implementing the Distributed Consensus Algorithm in a single file where all data is available in the same file.
2. Multi File Implementation: Implementing the Distributed Consensus Algorithm in a same device where all data are in different files.
3. Communication between Raspberry Pi: Establishing connection between two raspberry pi and Communicating between each other using different networking Protocols like UDP,TCP.
4. Consensus using LAN connection: Implementing the Consensus Algorithm for raspberry pi by setting up each raspberry pi as a node. Every nodes are connected through each other using ethernet (LAN).
5. Consensus using wireless connection: Implementing the Consensus Algorithm for raspberry pi by setting up each raspberry pi as a node. Every nodes are connected through each other using internet. All raspberry pi are not required to be in the same network or LAN in this case.
6. Single Raspberry Pi Control: The above algorithms are then implemented by only activating the coordinator node. The coordinator node sends the files to the nodes, runs them there, stores data for each iteration and sends them back to the coordinator.

The link for the Git hub repository that contains all the final versions of the project is [here](#).

3 Single File Implementation

In single File implementation we are already given with the values of each node in a single file. Hence, we go on computing every state using the node data and graph data. We are given the states of each node and adjacency matrix of each node. We then calculate the Degree matrix and Laplacian matrix and use the average consensus formula for computation.

$$D_{ii} = \sum A_{ij} \quad (1)$$

A is the adjacency matrix that contains the data of edges from each node to other. The degree matrix is then computed using the above equation.

$$L = D - A \quad (2)$$

The above equation is used in the computation of Laplacian matrix.

$$\dot{x} = -Lx. \quad (3)$$

The above computation is done in every iteration and the changes in the x values are then added as a fraction of a variable h to the actual states and this process is continued till the maximum number of desired iterations.

4 Multi File Implementation

In the upcoming sections we will be dealing with discrete time approach only. In multi file approach we will be having multiple text files in .txt format storing node data (state of each node). There will be a python file associated to every text file which can read and write the text file to make changes. In our approach we have a coordinator node which will send initiation message to all node file and the node files then get activated to perform iterations of distributed algorithm. The formula used at each node is given by the equation:

$$\dot{x}_i = \sum_{j \in N_i} (x_j - x_i) \quad (4)$$

In each node the following equation computes the change of state and this change of state will then be subtracted from it as a fraction of alpha will be repeated after equal intervals of time after receiving and sending data. The coordinator file contains all the data regarding number of iterations and neighbor data. The codes for the python files are available [here](#). The codes contain comments for explanation. In this implementation all the files are supposed to be present in the same folder.

5 Setup and Communication between Raspberry pi

Setup In order to setup the Raspberry pi for usage we first need to make sure we have the following devices:

1. Power Supply for Pi
2. Mouse
3. Keyboard
4. SD Card flashed with Pi OS
5. Monitor with HDMI Port
6. HDMI Cable

To flash SD cards with Pi OS we must have a laptop/computer with a SD card slot. We then need to download the Raspberry Pi imager from online. We then open the Imager application and select the respective requirements for the download after inserting the SD card into the slot. These include selecting the Raspberry Pi version we are using (in our case its 5), selecting the device to flash (SD card) 64-bit OS, etc... The imager will then start flashing the SD card. Once its done we can remove the SD card and insert it into the Raspberry Pi for usage. We need to connect mouse, keyboard, power supply and monitors appropriately for use. Once connected you can use the Pi like a computer. If you are a new user you will be asked to enter username, password, time zone, etc... We will be ready to use of Raspberry Pi after these steps. For the Raspberry Pi in the lab, the username is rasp1, rasp2, ... till rasp8. Passwords for all the Pi are 3.14 .

Communication We communicate between two Raspberry Pi either wireless (Connected to internet) or with wire. There are a set of commands or steps to be done before we can try to communicate between the Pi. We use the socket module in python for communication between two devices using either TCP or UDP protocol. Here in our project we use UDP protocol for our consensus. In this section we test both TCP and UDP. The setup for wired communication is explained below.

5.1 Wired Communication (LAN) with Ethernet Wires

Some devices/prerequisites required for the connections are, ethernet wires and switch for networking and communication. In order to communicate with each other in LAN with low latency with ethernet wires we need to download dhcpd5 module in terminal to make our ip address static. The steps to download dhcpd5 module are, Connect to internet in the Raspberry Pi. Then go to terminal and type the following commands one by one:

```
sudo apt update
sudo apt install dhcpd5
sudo systemctl enable dhcpd
sudo systemctl start dhcpd
```

For our module to work, we should reboot our Raspberry Pi. The loading screen and start screen might not be the same anymore after downloading dhcpd. The lightdm module which is responsible for the display of the Pi seems to fail after the download of dhcpd. We enter a command line interface instead of display interface now. We then need to type startx in the command line to go to display interface. This interface is not like the old interface anymore but we will be able to do all the tasks which was performed in the old interface.

```
startx
```

Now once this process is done to all Raspberry Pi we can go forward in connecting the ethernet wires from the Raspberry Pi to the switch and turn the power supply of switch on. After leaving the setup for a few minutes we can then go to terminal again and we can find the ip address of each device for ethernet connection using the command given below in terminal.

```
ifconfig
```

This commands give the ip address for all kinds of communication including WiFi networks, LAN and others. We must navigate to eth0 network in the output and we can obtain the ip address of the Raspberry Pi from there. This ip address remains the same forever if this Pi is connected to the given switch. We repeat the same for all the Raspberry Pi that we have and then we can proceed on to check the connection between the two using ping command in terminal. Let us consider two Pi for our time being and call them rasp1 and rasp2. Note down the ip address of both the Pi, let us call them ip1 and ip2. Once these two Pi are connected to the same switch we can test connection by going to rasp1 and typing the following command in terminal (IP Address of rasp2 is given at ip2).

```
ping ip2
```

If output shows packet transfer with a time taken of few milliseconds it means the Pi are connected. Whereas if we get output like request timed out or host unreachable it tells us that there are connectivity issues. We need to check ip addresses and retry the whole process again. If connections are successful we can go on for testing the TCP and UDP Connections. The code for the communication protocol testings are there in this [link](#). The ip addresses which we found from the ifconfig command will be used in code. So we must make sure to have the right ip addresses in the code before we start running the codes. We then run the codes to check for data transfer. Instructions are available in comments of the code. TCP protocol requires a permanent connection between the devices whereas for UDP connection packets are just sent from one device to an access point with the destination in the packet. TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are both transport layer protocols used for transmitting data over the Internet, but they handle data differently.

TCP (Transmission Control Protocol)

1. Connection-Oriented: Establishes a connection between the sender and receiver before data transmission begins.
2. Stream-Based: Data is sent as a continuous stream of bytes, and the recipient can reassemble these bytes into the original message.
3. Flow Control: Manages the rate of data transmission to prevent overwhelming the receiver.
4. High Latency: Data Transfer is slow due to the high error checking and control processes.

Example Uses: Web browsing (HTTP/HTTPS), email (SMTP), and file transfers (FTP)

UDP (User Datagram Protocol)

1. Connectionless: Does not establish a connection before sending data and does not guarantee delivery.
2. Datagram-Based: Sends data in discrete packets (datagrams), and each packet is independent of others.
3. No Flow Control: Does not manage the rate of data transmission.
4. Low Latency: Since there is no permanent connection required, data transfer happens quickly.

Example Uses: Streaming video/audio, online gaming, and DNS queries.

We use UDP Protocol in our project due to its connectionless nature and because of its low latency.

5.2 Wireless Communication on Internet

In wireless communication there is no necessity to have any ethernet or switches. We need to make sure that the devices are connected to internet. There are some setups in this again to be done for establishing communication. For us to establish communication through internet we setup a VPN so that data transfer is restricted in that VPN. Here in this section we use zero tier for VPN setup. The setup steps are explained later in the same subsection.

A VPN (Virtual Private Network) is a technology that creates a secure, encrypted connection over a less secure network, such as the Internet. This allows users to send and receive data as if their devices were directly connected to a private network.

Key Features of VPN

1. **Encryption:** VPNs encrypt your internet traffic, which means your data is scrambled and unreadable to anyone who might intercept it. This protects your privacy and sensitive information.
2. **Privacy:** By masking your IP address, a VPN helps hide your online activities from websites, ISPs (Internet Service Providers), and other entities. This can prevent tracking and profiling.
3. **Secure Access:** VPNs provide secure access to a private network from anywhere in the world. This is especially useful for businesses allowing remote employees to access company resources securely.
4. **Bypassing Restrictions:** VPNs can help bypass geographic restrictions and censorship by making it appear as though you are accessing the internet from a different location.
5. **Protection on Public Wi-Fi:** VPNs offer added security when using public Wi-Fi networks, protecting you from potential threats and hackers.

Here our main objective is to establish a secure access through any private from anywhere in the world.

Setup Instructions

1. Create an account in zerotier in any web browser (not necessarily in Raspberry Pi) and create a VPN network there.
2. Go into the network and turn the network to public and note down the network id of the network. This will be used for connectivity.
3. Download zerotier in the raspberry pi devices using the following command in terminal.

```
curl -s https://install.zerotier.com | sudo bash
```

4. Inorder to verify the installation we give the following command.

```
sudo systemctl status zerotier-one
```

5. To join to the network we then give the following command with the network id we noted down before.

```
sudo zerotier-cli join NETWORK_ID
```

6. These steps are sufficient to get connected to the VPN created. We then can obtain the ip address for each device using the ifconfig command and navigating to zero tier in the output.

```
ifconfig
```

Now to test the connectivity from one raspberry pi to other we repeat the ping test that was performed in wired connection using the new ip addresses. If we succeed in it we can move on to implement the connectivity using TCP and UDP using the same code which we used for wired connection with a modification in ip addresses. The latency is high in this because of wireless mode of communication.

6 Consensus using LAN connection

This section will repeat the same steps that was performed in Communication of Raspberry Pi section with multiple raspberry pi after configuring everything for wired data transfer. We first connect all the raspberry pi to the D-Link switch through ethernet. We then start configuring it to perform data transfer. We also will note all IPs and then use them for ping command for testing and also for the upcoming code.

In the consensus algorithm with we are implementing we will be wanting to install different python modules in the device. The command used to install any module in python is given as,

```
sudo apt install python3-{module name}
```

After giving these commands we will be prompted to confirm the installation by entering Y. If its done the installation will be done within a few seconds. For example, we will need to download pandas and paramiko module in the upcoming sections. So the commands to download them are,

```
sudo apt install python3-pandas  
sudo apt install python3-paramiko
```

Now in the consensus that I implemented, we have a coordinator file which is responsible to send all the important data like sleeptime, neighbor data, neighbor ip, neighbor ports and etc... Once the coordinator is being run in a file the coordinator sends initiation request to all other nodes along with the data required for running the files. Before

running the coordinator all the node files are supposed to be ran in their respective nodes. Node files tend to keep on waiting till the initiation message reaches them. Coordinator file then is ran sending initiation to all the nodes. Then the nodes start implementing the consensus iterations after sleeping for a uniform sleep time which we give. The sleep time is very necessary for synchronisation and it makes sure that no node over takes the other while running the iterations. During each iteration node data are sent to other nodes through UDP protocol. Nodes send information and wait for the information at the same time. Having high sleep time like 1s or 0.1 seconds allows us to perform around 1-10 iterations per second but these give accurate results. Decreasing sleep time to 0.01 seconds gives us 100 iterations per second but answer is precise enough for usage but not as precise as the previous ones. The more we decrease the sleep time the more deviation we get. Our consensus works better with around 0.0033 seconds which approximately ranges to 250-300 iterations per second giving us required results. Our algorithm does not fail when it does not receive a value from any neighbor. In order to make it robust the algorithm uses previous neighbor states which will be used to perform the iteration. Using previous states reduces the accuracy but it makes sure the algorithm does not stop or fail because of slow transmission from one node. This usage of previous states happens mostly when the sleep time is very low and does not happen for high sleep times. When sleep time goes below a threshold which is 0.0033 seconds here, the algorithm uses previous states again and again as the sleep time gets lesser than the transmission time causing huge deviations. The code for the consensus is available in the given [link](#). The node number and nodes values can be altered on own will. The node.py files for each node differ from each other only in the node-id and ip address. All other data come from coordinator, hence changes are to be done in coordinator if number of nodes are changed and new nodes must be able to communicate. The state values are linked with text files in each node. So we should go to each and every node file in order to make change in state values.

7 Consensus with Wireless connections

This section is no different from the previous section except for us ensuring to use the correct zero tier IP addresses in our codes. Theoretically there is not much of a difference between the previous section and this section. Implementation takes time as zerotier is supposed to be installed in every node and IPs of each nodes are to be noted and changed in the code file. The procedures of finding IP, installing zero tier, etc are all given in the above sections, please refer it to establish connections. The code for LAN network works flawlessly with Internet Consensus when the correct IP addresses are used. The link for the code is same as for the above section.

8 Single Raspberry Pi Control

Since its tedious for any one to setup every node and then run the code in every Raspberry Pi using few monitors and desktops we try to send the code and run the code from a single raspberry pi to other raspberry pi with only giving the power supply to the raspberry pi. But in order to achieve this there are some predefined setup which is supposed to be done for one raspberry pi to send and run code in other raspberry pi. We will be using ssh and scp commands along with paramiko module in python for this purpose. The steps

and the things that are supposed to be done are mentioned below.

1. We must make sure that all the python modules that are used in our node.py files are installed already. If not they are supposed to be installed in every Raspberry Pi.
2. All the nodes that are to be used should have gone through all the process like installing dhcpcd5 and so on. Make sure to all node down the username and passwords of respective raspberry pi as they would be required to establish ssh connections.
3. We should now enable ssh in every raspberry pi so that we can use ssh and scp between the devices. We do that by the following steps.

(a) Enter the following command in terminal.

```
sudo raspi-config
```

(b) Now use arrow keys to select 'INTERFACE OPTIONS' and press enter.

(c) Select SSH and press enter.

(d) Then select YES to enable SSH server.

(e) use arrow keys to go to finish and click enter. Tab key also can be used to select finish. Once this is done you will go back to terminal and our SSH server would be enabled.

4. Now we generate SSH keys in every device using the command.

```
ssh-keygen -t rsa -b 2048
```

We get consecutive messages like

```
Enter file in which to save the key (/home/your_username/.ssh/id_rsa):  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:
```

Press enter for every prompts like these above. Our SSH keys will then be ready with a message.

5. Now connect every raspberry pi including coordinator pi to the D-Link switch with ethernet cable. Now make sure to note the ip addresses of eth0 for every node and use them for the upcoming processes.
6. Do this to every raspberry pi in order to generated ssh keys for ssh connectivity. We will be establishing this ssh connectivity between nodes and coordinator and vice versa only. We do not establish any ssh connectivity between nodes because there is no necessity between nodes to send or run files. The access for coordinator to nodes is established by the following command.

```
ssh-copy-id node_username@node_ip
```

Replace the above code's node username with the nodes actual username and node ip with the nodes ip address. This command will be responsible to copy the rsa keys of the coordinator to the nodes. The prompts that will be asked in return are,

```
The authenticity of host 'node_ip (node_ip)' can't be established.  
ECDSA key fingerprint is SHA256:XXXXXXXXXXXXXXXX.  
Are you sure you want to continue connecting (yes/no)?
```

Enter yes and then we will be prompted with,

```
node_username@node_ip's password:
```

Enter the node's password and then the RSA keys of coordinator will be copied to the node and the output in the terminal will be like,

```
Number of key(s) added: 1
```

```
Now try logging into the machine, with:  "ssh 'node_username@node_ip'"  
and check to make sure that only the key(s) you wanted were added.
```

If we get this message this means we have ssh control from coordinator to nodes, that is, the nodes can now be controlled completely from coordinator. The purpose of doing this is to send the node.py files from coordinator to nodes. To check if the ssh connections have established or not we give the following command.

```
ssh node_username@node_ip
```

7. We then perform the same above operations for nodes, so that nodes can now access coordinator. This is done so that nodes can send the iterations results csv back to coordinator.
8. This process is to be done between every node and coordinator. By this step we would have established connections with every node and coordinator and vice versa.

After this step there is no necessity to connect any node raspberry pi to monitor. We must just make sure give power supply to other raspberry pi and let it turn on for a few minutes and then we can run the code from coordinator. Connecting the coordinator raspberry pi to coordinator will be sufficient as we now will be sending all the files from coordinator to the nodes and run them. The results then are collected back from the nodes by the coordinator and the graphs are then plotted. The implementation and execution remains the exact same except for the usage of ssh and scp modules for data transfer between the Pi. The code for the implementation is there in this [link](#).

The code contains coordinator.py file, sender.py file and Node files. The sender.py files is responsible to send the Node files to the respective nodes. Before we run the sender.py we must create the respective node files in the folder where we have coordinator and sender and we must ensure that the ip addresses used in each python file are correct and appropriate. After creating the correct nodes files in the names N1.py, N2.py and so on and after verify the node ids and ip addresses we can run the coordinator file to send these files to the nodes and run them. Its then preferred to wait for a minute or two before starting the coordinator code which then initiates the consensus process. Its better to wait for a few minutes after running the sender.py before running coordinator.py so that the we give the devices the time for the code to go to other nodes and get written and run over there. Then we run the coordinator.py file which starts the consensus process. Once the consensus process is over, we receive done message from all nodes along with the csv file of data containing the values at each iteration. Once the done message is received from all the nodes our coordinator file then plots the graph for all the values received from the nodes in a single plot. This ends the consensus process using a single raspberry pi bringing us to the end of the project.